

**UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY**

—o0o—

**COURSE PROJECT REPORT
NETWORK PROGRAMMING**

PROJECT: VIDEO STREAMING USING IP MULTICAST



Group Members:
Đặng Võ Hồng Phúc
Trịnh Chấn Duy

Student IDs:
23120155
23120419

Ho Chi Minh City, May 2026

Contents

1	Introduction and Specifications	1
1.1	Project Overview	1
1.2	Group Members and Task Allocation	1
2	System Architecture and Protocol Design	2
2.1	Multicast Streaming Paradigm	2
2.2	Custom Packet Format	2
2.3	Concurrent Clients and Loss Detection Algorithm	3
3	Code Implementation Details	4
3.1	Custom Packet Encoding/Decoding (CustomPacket.py)	4
3.2	Server Implementation (Server.py)	4
3.3	Client Implementation and GUI Threading (Client.py)	5
4	Verification and Experimental Results	6
4.1	Server Operation and Initialization	6
4.2	Client Playback and Concurrency	6
4.3	Statistics, Loss Detection, and IGMP Leave	7

Chapter 1

Introduction and Specifications

1.1 Project Overview

This laboratory report provides a comprehensive examination of the architecture, design, and implementation of a multicast video streaming application over IP. The developed system operates on a direct Server-to-Multicast paradigm where multiple clients join a multicast group to watch a shared stream without any individual, stateful connection overhead.

The core objective of the project is to implement a streamlined, robust multicast framework that bypasses complex legacy protocols. The key technical features implemented and discussed in this report include:

- IP Multicast Architecture:** The server continuously broadcasts Motion JPEG (MJPEG) video frames to a specific UDP Multicast group (239.1.1.1:5004), eliminating the need for RTSP, RTP, TCP/UDP selection, or separate control/data channels. By leveraging IGMP (Internet Group Management Protocol), the network routers handle the duplication of packets to end clients, maintaining $O(1)$ server bandwidth complexity regardless of client count.
- Custom Packetization Protocol:** Standard RTP introduces significant byte overhead and parsing complexity. To optimize delivery, a minimal 10-byte custom packet header mechanism was designed to encapsulate frames, maintaining integrity while allowing for sequence tracking and packet loss detection.
- Real-Time Playback and Loss Detection:** The client concurrently listens to the multicast group, decodes received packets, tracks skipped frames for dynamic loss calculation, and renders the video in real-time utilizing a Tkinter GUI.
- Automatic Loop and Clean Group Management:** The server resets playback transparently upon video completion. The client correctly manages its network footprint by executing an IGMP Join upon execution and an IGMP Leave upon termination.

1.2 Group Members and Task Allocation

The project was executed collaboratively by the following team members. Both members contributed extensively to the design, implementation, and testing phases.

Full Name	Student ID	Email
Đặng Võ Hồng Phúc	23120155	23120155@student.hcmus.edu.vn
Trịnh Chấn Duy	23120419	23120419@student.hcmus.edu.vn

Chapter 2

System Architecture and Protocol Design

2.1 Multicast Streaming Paradigm

Unlike unicast models where a server must duplicate bandwidth for each connected client, IP Multicast operates by sending a single stream into the network. Routers and switches then duplicate the packets only where necessary, reaching all clients subscribed to the group.

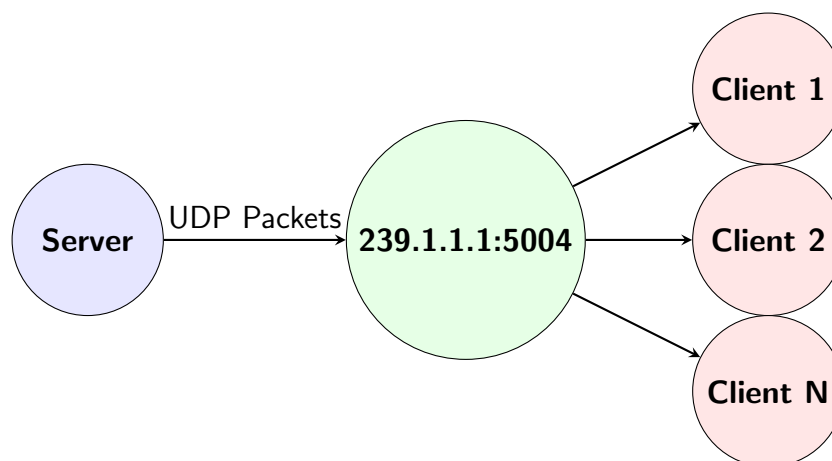


Figure 2.1: Server-Multicast Group Architecture Diagram

The server does not know who the clients are, nor does it keep any TCP connection open. It blindly packetizes the MJPEG video and transmits datagrams at approximately 20 frames per second (50ms per frame). Because the server does not handle client acknowledgments or connection states, its CPU and Memory utilization remain flat even if thousands of clients join the stream.

2.2 Custom Packet Format

Instead of relying on standard but bloated protocols like RTP, a **Custom Packet Format** was devised. This provides exactly the information the client needs to reassemble the data and calculate packet loss without any unnecessary overhead.

The header format consists of exactly 10 bytes arranged in Big-Endian (Network) byte order:

- **Magic Number (2 bytes):** 0x1234, used to verify that the incoming packet actually belongs to our streaming application and isn't random network noise.

- **Frame Number (4 bytes):** An unsigned integer incrementing continuously. Essential for the client to detect missing packets and calculate the loss rate.
- **Payload Length (4 bytes):** An unsigned integer dictating the size of the MJPEG frame appended immediately following the header.

By limiting the header to 10 bytes, the payload to header ratio is vastly improved compared to a 12-byte RTP header plus potential extension headers, allowing for maximum data throughput.

2.3 Concurrent Clients and Loss Detection Algorithm

Since UDP Multicast provides no guarantee of delivery, packets may be dropped by the network (due to router congestion or buffer overflow). The client tracks the Frame Number extracted from the Custom Packet header.

The algorithm operates as follows:

1. When the first packet arrives, the client sets `expected_frame` to `frame_number + 1`.
2. For subsequent packets, if `frame_number > expected_frame`, the difference represents the exact number of frames lost during transit.
3. The client accumulates `lost_frames` and updates the `expected_frame` variable.
4. A real-time UI widget continuously updates the total received packets, lost packets, and the loss percentage formula:

$$\text{Loss Rate (\%)} = \frac{\text{lost_frames}}{\text{lost_frames} + \text{received_frames}} \times 100$$

Multiple clients can operate on the same machine or network independently. They will each calculate statistics based on the exact moment they executed the IGMP join command, demonstrating accurate per-client network diagnostics.

Chapter 3

Code Implementation Details

This chapter highlights the core components implemented in Python, particularly focusing on the usage of the socket and struct libraries for low-level network manipulation.

3.1 Custom Packet Encoding/Decoding (CustomPacket.py)

The CustomPacket.py module uses Python's struct library for rapid binary serialization using network byte order (!). The H format character maps to a 2-byte unsigned short (Magic Number), and the two I characters map to 4-byte unsigned integers (Frame Number and Payload Length).

```
1 import struct
2
3 class CustomPacket:
4     MAGIC = 0x1234
5     HEADER_FORMAT = "!HII"
6     HEADER_SIZE = struct.calcsize(HEADER_FORMAT)
7
8     @staticmethod
9     def encode(frame_num, payload):
10         header = struct.pack(CustomPacket.HEADER_FORMAT,
11                               CustomPacket.MAGIC, frame_num, len(payload))
12         return header + payload
13
14     @staticmethod
15     def decode(data):
16         if len(data) < CustomPacket.HEADER_SIZE:
17             return None, None
18         header = data[:CustomPacket.HEADER_SIZE]
19         magic, frame_num, length = struct.unpack(CustomPacket.HEADER_FORMAT,
20 header)
21         if magic != CustomPacket.MAGIC:
22             return None, None
23         payload = data[CustomPacket.HEADER_SIZE:CustomPacket.HEADER_SIZE+length]
24         return frame_num, payload
```

Listing 3.1: CustomPacket.py - Header Structure

3.2 Server Implementation (Server.py)

The Server script binds a UDP socket, applies the IP Multicast Time-To-Live (TTL) configuration (IP_MULTICAST_TTL), and streams the MJPEG file frame-by-frame. A TTL of 2 ensures that the stream

can pass through the local subnet to adjacent routers, though it can be adjusted for broader WAN distribution. Upon reaching the file's end, it automatically seeks back to the 0th byte.

```
1 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
2 sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, 2)
3
4 while True:
5     frame = video_stream.nextFrame()
6     if frame is None:
7         video_stream.reset() # Replay video automatically
8         continue
9
10    packet = CustomPacket.encode(video_stream.frameNum, frame)
11    sock.sendto(packet, (MULTICAST_GROUP, MULTICAST_PORT))
12    time.sleep(0.05) # Delay to enforce ~20 FPS pacing
```

Listing 3.2: Server.py - Core Multicast Loop

3.3 Client Implementation and GUI Threading (Client.py)

The Client script uses Tkinter to construct the user interface. It binds a UDP socket with the `SO_REUSEADDR` flag (and `SO_REUSEPORT` on supported OSs), allowing multiple clients to run simultaneously on the same host and listen to the same port. It then issues an IGMP Join request via `IP_ADD_MEMBERSHIP`.

Because Tkinter's `mainloop()` is blocking and must run on the main thread, the networking `receive_loop` is offloaded to a background Daemon thread.

```
1 self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
2 self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
3 self.sock.bind(('', MULTICAST_PORT))
4
5 # Issue IGMP Join for Multicast Group 239.1.1.1
6 mreq = socket.inet_aton(MULTICAST_GROUP) + socket.inet_aton('0.0.0.0')
7 self.sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
8
9 # Start background thread to avoid freezing GUI
10 self.receive_thread = threading.Thread(target=self.receive_loop)
11 self.receive_thread.daemon = True
12 self.receive_thread.start()
```

Listing 3.3: Client.py - Threading and Socket Setup

The receive loop extracts the payload and schedules the UI update using `master.after(0, ...)` to ensure thread safety when modifying the Tkinter Label widget:

```
1 data, _ = self.sock.recvfrom(65536) # Max UDP Size buffer
2 frame_num, payload = CustomPacket.decode(data)
3
4 if frame_num is not None:
5     # Update loss tracking logic...
6     self.received_frames += 1
7
8     # Safely dispatch image rendering back to the Main GUI Thread
9     self.master.after(0, self.update_display, payload)
10    self.master.after(0, self.update_stats)
```

Listing 3.4: Client.py - Packet Reception and Thread-Safe UI Update

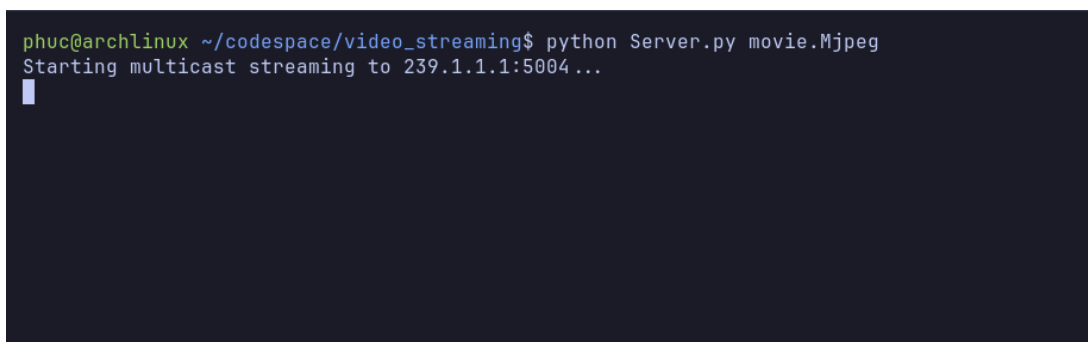
Chapter 4

Verification and Experimental Results

This chapter provides visual evidence of the system operating successfully under specified constraints.

4.1 Server Operation and Initialization

The server successfully initializes using the command `python Server.py movie.Mjpeg`. Without waiting for any client handshakes, the server begins continuous injection of video packets into the 239.1.1.1 multicast group at port 5004. This verifies the stateless architecture required by the specifications.

A terminal window with a dark background. The prompt is 'phuc@archlinux ~/codespace/video_streaming\$'. The command entered is 'python Server.py movie.Mjpeg'. The output is 'Starting multicast streaming to 239.1.1.1:5004...' followed by a cursor on a new line.

```
phuc@archlinux ~/codespace/video_streaming$ python Server.py movie.Mjpeg
Starting multicast streaming to 239.1.1.1:5004...
█
```

Figure 4.1: Server terminal successfully running and broadcasting

4.2 Client Playback and Concurrency

Multiple instances of the client can be launched concurrently using `python Client.py`. Each instance independently joins the multicast group. Upon launch, the GUI displays a waiting state until the first video packets arrive, after which real-time playback initiates instantly.

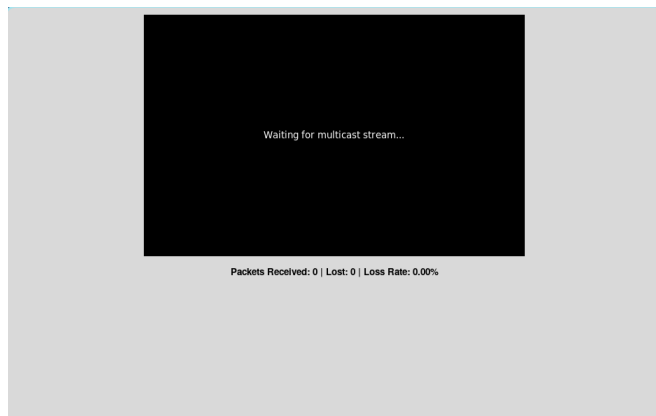


Figure 4.2: Client GUI in waiting state

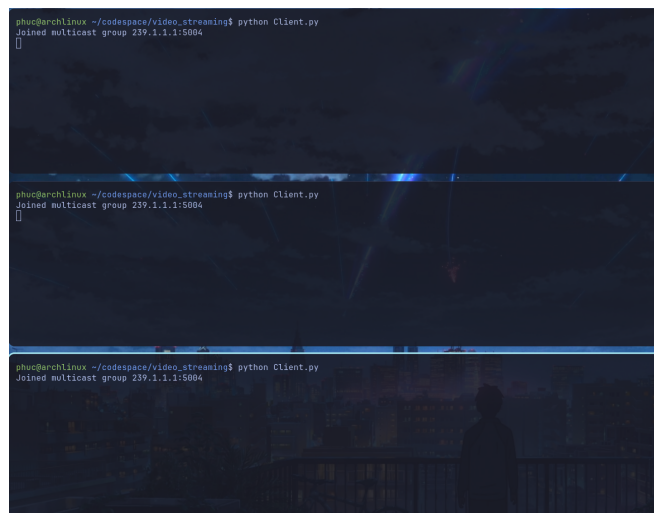


Figure 4.3: Client GUI displaying the video stream

This sequential rendering confirms that the application effectively leverages IP Multicast's inherent scalability capabilities to serve unlimited concurrent watchers with zero additional overhead on the server.

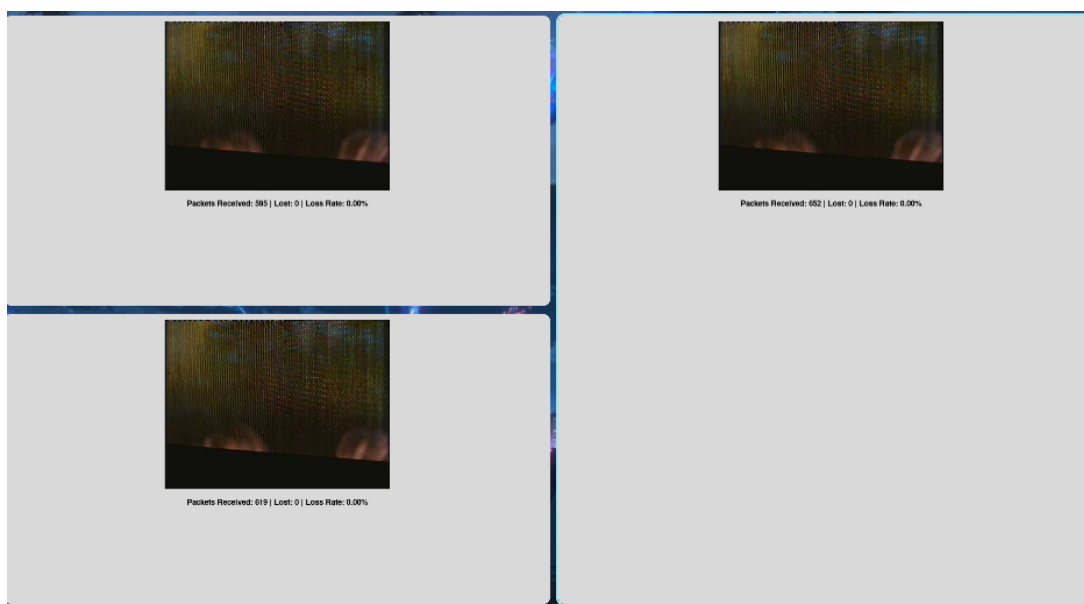


Figure 4.4: Multiple Client windows running simultaneously on the same screen

4.3 Statistics, Loss Detection, and IGMP Leave

During execution, the loss detection logic performs exceptionally. Each UI instance displays its own live counter. If a network disruption occurs (or frames are deliberately dropped by temporarily suspending the server/client), the client accurately identifies the skipped frame numbers based on our Custom Packet header and updates the "Packets Lost" and "Loss Rate %" counters dynamically.



Figure 4.5: Client GUI highlighting the "Packets Lost" and "Loss Rate" statistics

When a client application is closed via the window manager, the application intercepts the WM_DELETE_WINDOW event and successfully executes IP_DROP_MEMBERSHIP on the socket. This forces the OS to send an IGMP Leave message, properly exiting the group and preventing continued unnecessary network traffic caching by local routers.